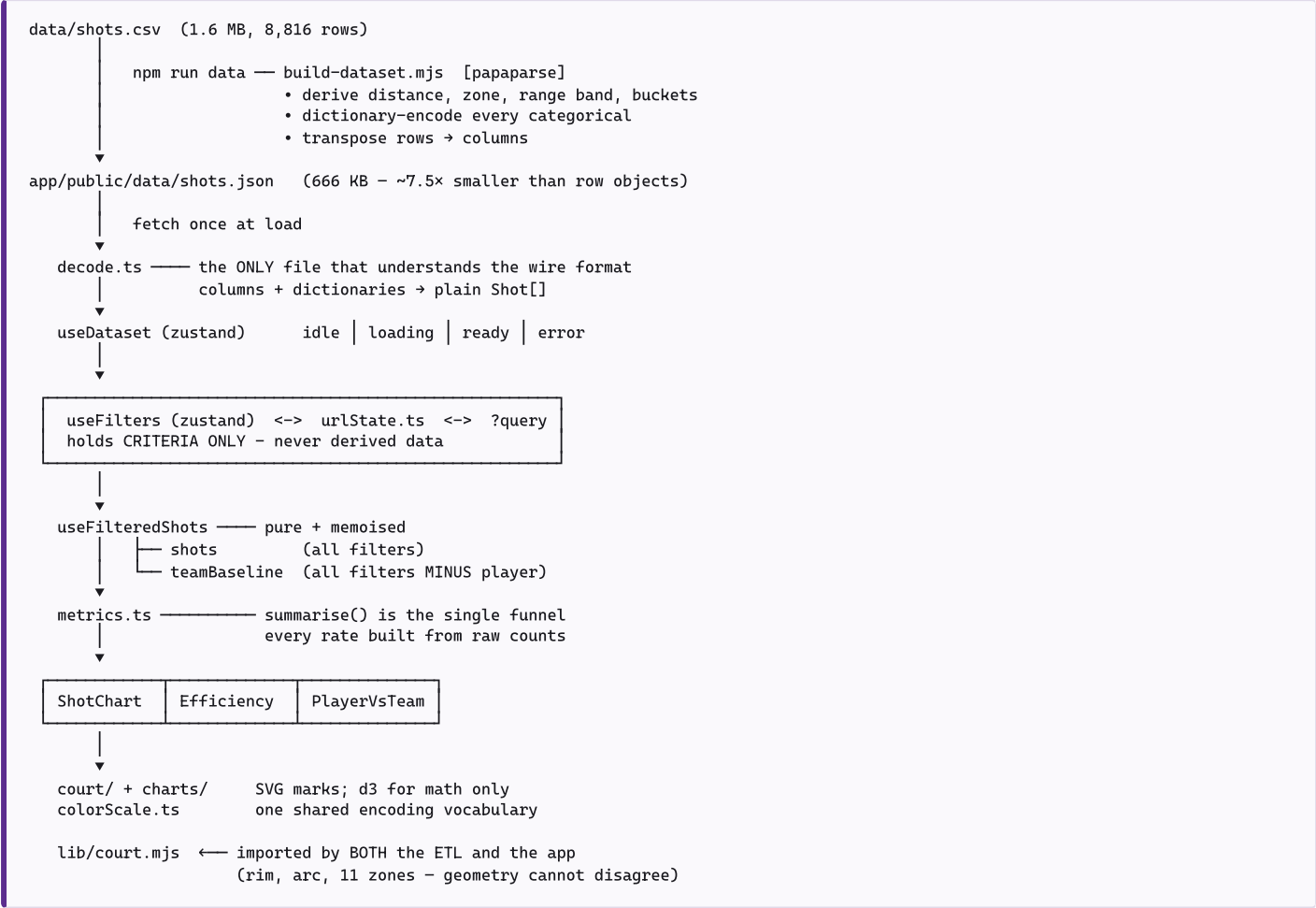


Architecture

One idea runs through the whole thing: **derive everything once at build time, so the browser does analytics instead of data-prep.**

~4,200 lines of TS/TSX across 36 files, plus ~2,100 lines of CSS.

Data flow



Why each seam exists

Seam	Reason
Build-time ETL	Distance, zone, buckets and flags are derived once, so filtering stays in single-digit milliseconds.
Columnnar + dictionary encoding	1.6 MB CSV → 666 KB JSON. Categoricals become integers; rows become columns.
decode.ts as the only reader	Exactly one file knows the compact format. Change the encoding, change one file.
court.mjs shared by ETL and app	The zone a shot was classified into and the polygon drawn on screen come from the same constants. The picture cannot disagree with the numbers.
Store holds criteria only	No cached derived data, so nothing can go stale.
Zustand over Context	Components subscribe to slices. With Context, toggling one filter would re-render thousands of SVG marks.
teamBaseline alongside shots	Same filters minus the player selection, so "player vs team" always compares under identical context — never against an unfiltered season total.
summarise() as single funnel	Rates computed from raw counts, never averaged, so a total is the same however the slice is composed.

Seam	Reason
d3 for math, not rendering	<code>scaleThreshold</code> / <code>scaleQuantize</code> / <code>scaleSqrt</code> for encoding, <code>hexbin</code> for the honeycomb, <code>quadtrees</code> for hover. Marks are hand-rendered SVG.
No input validation	The brief states the data is clean and asks for a simple solution. <code>build-dataset.mjs</code> marks where guards would go if that changed.

Key modules

File	Role
<code>scripts/build-dataset.mjs</code>	ETL: CSV → columnar dictionary-encoded JSON
<code>lib/court.mjs</code>	Court geometry + shot classification. Single source of truth.
<code>lib/court.ts</code>	Typed surface over <code>court.mjs</code> for the app
<code>lib/metrics.ts</code>	<code>summarise</code> , <code>groupBy</code> , <code>compareToBaseline</code> , <code>pointsAddedVsBaseline</code>
<code>lib/colorScale.ts</code>	<code>efficiencyColor</code> , <code>volumeColor</code> , <code>radiusForVolume</code>
<code>data/decode.ts</code>	Columnar payload → <code>Shot[]</code>
<code>state/useFilters.ts</code>	Filter criteria store, hydrates from URL
<code>state/useFilteredShots.ts</code>	Pure memoised derivation → <code>shots</code> + <code>teamBaseline</code>
<code>state/urlState.ts</code>	Filters ⇌ query string
<code>components/court/</code>	<code>CourtSvg</code> , <code>ZoneLayer</code> , <code>HexLayer</code> , <code>ShotLayer</code>
<code>components/charts/</code>	<code>BarChart</code> , <code>DivergingBarChart</code> , <code>MatrixChart</code> , <code>ScatterChart</code>

Metrics

- **eFG%** = (FGM + 0.5 × 3PM) / FGA
- **PPS** = points / attempts (exactly 2 × eFG%, in a unit coaches argue in)
- **Points added** = Σ (player PPS – team PPS) × attempts, per zone
- **Low sample** = under 25 attempts. Marked and hatched, never hidden.